

Лекция 8. Android Garbage collector

- Алгоритмы GC, типы ссылок.
- Создание игровых приложений в платформе Unity 3d.
- Базовые концепции, структура проекта.

Алгоритмы Garbage Collector и типы ссылок

Garbage Collector (GC) в Android — это механизм, автоматически освобождающий память, занятую недостижимыми объектами. Его работа основана на двух ключевых этапах: Mark (пометка достижимых объектов от корневого множества — активных потоков, статических переменных, локальных переменных в стеке) и Sweep (очистка памяти, занятой недостижимыми объектами). Эффективность GC напрямую влияет на производительность, особенно в ресурсоёмких приложениях, таких как игры.

В ранних версиях Android (Dalvik) GC использовал Mark-Sweep с Stop-The-World (STW) — при каждом цикле сборки GC останавливал выполнение приложения, вызывая паузы. Например, если игра на Unity динамически создавала временные объекты (например, частицы или строки), это могло привести к частым STW-паузам, что проявлялось как заикания.

С переходом на ART (Android Runtime) в Android 5.0+ GC стал использовать Concurrent Mark-Sweep, где фаза Mark выполняется параллельно с работой приложения, минимизируя STW-паузы. Это особенно важно для игр, где даже миллисекундные задержки критичны.

В Android 8.0+ появился Concurrent Copying, который уменьшает паузы при сборке молодого поколения (Young Generation). В Android 12+ добавлен Userfaultfd-based GC, использующий возможности ядра Linux для ещё более плавной работы с большими кучами.

Типы ссылок и их влияние на GC следующие:

- Strong Reference — стандартные ссылки, которые блокируют сборку мусора, пока объект используется. Например, в Unity основные игровые

объекты (игрок, камера) хранятся через сильные ссылки, чтобы GC не удалил их случайно.

- Soft Reference — идеальна для кэширования. GC удаляет такие объекты только при нехватке памяти, например, текстуры фоновых уровней, которые можно перезагрузить.
- Weak Reference — не препятствует сборке мусора. Используется для слушателей событий, которые должны автоматически отписываться при уничтожении объекта.
- Phantom Reference — самая «слабая» ссылка, позволяющая отслеживать финализацию объекта через ReferenceQueue. Редко применяется в обычных приложениях.

Пример 1: Кэширование через WeakReference

```
class EventManager {  
    private val listeners = mutableListOf<WeakReference<EventListener>>()  
  
    fun addListener(listener: EventListener) {  
        listeners.add(WeakReference(listener))  
    }  
    fun triggerEvent() {  
        listeners.forEach { weakRef ->  
            weakRef.get()?.onEventTriggered()  
        }  
    }  
}
```

Здесь EventManager хранит слушатели в WeakReference, чтобы GC мог очистить их, если объект EventListener больше не используется. Это предотвращает утечки памяти, например, когда слушатель уничтожается, но остаётся в списке.

Пример 2: Использование SoftReference для кэширования текстур

```
class TextureCache {  
    private val cache = HashMap<String, SoftReference<Bitmap>>()  
  
    fun getTexture(name: String): Bitmap? {  
        val cached = cache[name]?.get()  
        return if (cached != null) {  
            cached // Возврат кэшированной текстуры  
        }  
    }  
}
```

```

    } else {
        val newTexture = BitmapFactory.decodeResource(resources,
resources.getIdentifier(name, "drawable", packageName))
        cache[name] = SoftReference(newTexture)
        newTexture
    }
}
}

```

Здесь `SoftReference` позволяет кэшировать текстуры, освобождая их только при нехватке памяти. Это снижает риск `OutOfMemoryError`, особенно в Unity, где текстуры часто грузятся и выгружаются динамически.

Создание игровых приложений в Unity 3D

Unity строит игру на основе игрового цикла, где каждый кадр состоит из нескольких этапов:

- Ввод и события: Обработка нажатий клавиш, тач-событий, сенсорных данных.
- Обновление сценария: Вызов `Update()` у всех активных скриптов.
- Физика: Обновление положения объектов через `FixedUpdate()` (для физических расчётов).
- Отрисовка: Рендеринг кадра на экране.

Unity использует двухуровневое управление памятью:

- Управляемая куча (Managed Heap): Хранит объекты C# (`GameObject`, `MonoBehaviour`, `List`, пользовательские классы). Управляется GC Unity (Boehm GC или IL2CPP).
- Нативная куча (Native Heap): Хранит ресурсы Unity (текстуры, меши, звуки) и нативные плагины. Управляется ART GC.

Метод `Update()` — это один из ключевых методов жизненного цикла в Unity, который автоматически вызывается движком один раз за кадр. Это означает, что код, написанный внутри `Update()`, выполняется повторно с частотой, равной частоте кадров вашей игры (обычно 30–60 раз в секунду), что делает его уязвимым для проблем, связанных с управлением памятью.

Каждый раз, когда вы создаете новый объект внутри Update() (например, Vector3, RaycastHit или строку через конкатенацию), Unity выделяет для него память в управляемой куче. Эта память со временем освобождается Garbage Collector (GC), но сам процесс сборки мусора требует ресурсов и может вызвать паузы в выполнении кода, которые проявляются как заикания в игре.

Пример 3: Снижение аллокаций через кэширование

```
public class PlayerController : MonoBehaviour {
    private Rigidbody rb;

    void Start() {
        rb = GetComponent<Rigidbody>(); // Однократное получение компонента
    }

    void Update() {
        if (Input.GetKeyDown(KeyCode.Space)) {
            rb.AddForce(Vector3.up * 10f); // Использование кэшированного компонента
        }
    }
}
```

Здесь вызов GetComponent<Rigidbody>() в Start() исключает аллокации в Update(). Это снижает нагрузку на GC, так как GetComponent() возвращает существующую ссылку, а не создаёт новый объект.

Пример 4: Пул объектов для частиц

```
public class ParticlePool : MonoBehaviour {
    public GameObject particlePrefab;
    private Queue<GameObject> pool = new Queue<GameObject>();

    void Start() {
        for (int i = 0; i < 100; i++) {
            var particle = Instantiate(particlePrefab);
            particle.SetActive(false);
            pool.Enqueue(particle);
        }
    }

    public GameObject GetParticle() {
        if (pool.Count > 0) {
            var particle = pool.Dequeue();
            particle.SetActive(true);
        }
    }
}
```

```

        return particle;
    } else {
        return Instantiate(particlePrefab); // Резервный вариант
    }
}
public void ReturnToPool(GameObject particle) {
    particle.SetActive(false);
    pool.Enqueue(particle);
}
}

```

Пул объектов исключает частые вызовы `Instantiate()` и `Destroy()`, которые провоцируют GC. В примере 100 частиц создаются один раз, а затем переиспользуются. Это снижает аллокации и предотвращает паузы GC, обеспечивая плавную анимацию.

Пример 5: Использование структур вместо классов

```

struct ParticleData {
    public Vector3 position;
    public Vector3 velocity;
}
public class ParticleSystem : MonoBehaviour {
    private List<ParticleData> particles = new List<ParticleData>();

    void Start() {
        for (int i = 0; i < 1000; i++) {
            particles.Add(new ParticleData { position = Vector3.zero, velocity =
Random.insideUnitSphere });
        }
    }
    void Update() {
        for (int i = 0; i < particles.Count; i++) {
            particles[i] = particles[i] with { position += velocity * Time.deltaTime };
        }
    }
}

```

Структуры (`struct`) хранятся на стеке, а не в управляемой куче. В примере `ParticleData` обновляется без создания новых объектов, что снижает нагрузку на GC. Это особенно эффективно для временных данных, таких как движение частиц.

Базовые концепции и структура проекта Unity

Проект Unity состоит из сцен, игровых объектов, компонентов и ассетов . Правильная организация структуры проекта напрямую влияет на управление памятью и GC. Ключевые элементы Unity следующие:

- Сцена (Scene): Единица контента, содержащая иерархию GameObject.
- GameObject: Контейнер сущностей (игрок, стена, свет).
- Компонент (Component): Функциональность GameObject (Transform, Renderer, Collider).
- Скрипты (Script): Код на C#, реализующий логику.
- Префаб (Prefab): Шаблон GameObject, позволяющий создавать экземпляры через Instantiate().
- Ассеты (Asset): Все ресурсы проекта (текстуры, модели, звуки).

Структура папок проекта следующая:

- Assets: Корневая папка, где Unity отслеживает все ресурсы.
- _Scripts: Скрипты, разделённые на логические подпапки (Player, UI, Managers).
- Scenes: Файлы сцен (.unity).
- Prefabs: Префабы, используемые для создания объектов.
- Art / Graphics: Текстуры, спрайты, материалы.
- Models: 3D-модели.
- Animations: Анимации и контроллеры.
- Audio: Звуковые файлы.
- Resources: Ресурсы, загружаемые через Resources.Load().
- Plugins: Нативные библиотеки (.so для Android, .a для iOS).
- StreamingAssets: Файлы, доступные через Application.streamingAssetsPath.
- ProjectSettings: Настройки проекта (качество, ввод, теги).
- Packages: Управляемые через Package Manager пакеты (URP, Input System).

- Library: Автоматически генерируемый Unity кэш.

Рассмотрим следующий пример, в котором реализуется асинхронная загрузка и выгрузка текстуры с помощью системы Addressables, являющейся частью Unity и предназначенной для эффективного управления ресурсами. Основная цель — минимизировать нагрузку на память и исключить утечки, особенно при работе с большими ассетами, такими как текстуры или 3D-модели.

Пример 6: Использование Addressables для контролируемой загрузки ассетов:

```
using UnityEngine.AddressableAssets;
using UnityEngine.ResourceManagement.AsyncOperations;

public class AssetLoader : MonoBehaviour {
    // Хранит ссылку на асинхронную операцию загрузки текстуры
    private AsyncOperationHandle<Texture2D> textureHandle;

    // Метод для запуска асинхронной загрузки текстуры по ключу
    void LoadTexture(string assetKey) {
        // Загрузка текстуры по ключу assetKey
        Addressables.LoadAssetAsync<Texture2D>(assetKey).Completed +=
OnTextureLoaded;
    }
    // Обработчик завершения загрузки текстуры
    void OnTextureLoaded(AsyncOperationHandle<Texture2D> handle) {
        // Проверка успешности загрузки
        if (handle.Status == AsyncOperationStatus.Succeeded) {
            // Сохраняем handle для последующей выгрузки
            textureHandle = handle;
            // Применяем текстуру к материалу объекта
            GetComponent<Renderer>().material.mainTexture = handle.Result;
        }
    }
    // Метод для выгрузки текстуры из памяти
    void UnloadTexture() {
        // Освобождение текстуры, чтобы предотвратить утечки
        Addressables.Release(textureHandle);
    }
}
```

Рассмотрим, что происходит в вышеописанном коде:

1) Асинхронная загрузка через LoadAssetAsync().

- Метод `Addressables.LoadAssetAsync<Texture2D>(assetKey)` иницирует неблокирующую загрузку текстуры по указанному ключу (`assetKey`). Это означает, что загрузка происходит в фоне, не останавливая выполнение игры.
- `AsyncOperationHandle<Texture2D>` — это объект, который отслеживает статус операции: загружена ли текстура, возникла ли ошибка или процесс завершён. Он также содержит результат (`handle.Result`), который становится доступным после успешной загрузки.

2) Обработка завершения загрузки в OnTextureLoaded()

- Метод `OnTextureLoaded` подписывается на событие `Completed` асинхронной операции. Это гарантирует, что код внутри `OnTextureLoaded` выполнится только после завершения загрузки, а не сразу, что избавляет от блокировок и пауз в игре.
- Проверка `handle.Status == AsyncOperationStatus.Succeeded` обеспечивает безопасность: текстура применяется к материалу только в случае успеха.
- `textureHandle` сохраняется, чтобы в дальнейшем можно было явно освободить ресурсы через `Addressables.Release()`.

3) Выгрузка текстуры через Addressables.Release()

- Метод `UnloadTexture()` вызывает `Addressables.Release(textureHandle)`, который явно освобождает текстуру из нативной памяти. Это критично для мобильных устройств, где память ограничена.
- В отличие от `Resources.UnloadUnusedAssets()`, который работает неопределённо (GC может не освободить память даже при отсутствии ссылок), `Addressables.Release()` гарантирует, что текстура будет удалена из памяти немедленно, как только вызов будет выполнен.

Таким образом, использование Addressables вместо Resources.Load() и Destroy() формирует основу для эффективного управления памятью в Unity, снижая частоту работы GC и улучшая отзывчивость.